Data Structure and Algorithm

COURSE CODE: CS-201

TOPIC: INFIX TO POSTFIX CONVERSION- STACKS

INSTRUCTOR: HABIBA ARSHAD

Goal for today

- Reversal of Strings
- Infix to postfix conversion using stack
- evaluation of postfix expression

Practice Task

Write an algorithm to perform ‘push’ and ‘pop’ operations in Stack and by considering the following stack of city names, show the pictorial representation of stack as the below mentioned operations take place:

STACK: London, Berlin, Rome, Paris, _____, _____

Operations:

a. PUSH (STACK, Athens)
b. POP (STACK, ITEM)
c. POP (STACK, ITEM)
d. PUSH (STACK, Madrid)
e. PUSH (STACK, Moscow)
f. POP (STACK, ITEM)

Application of stacks

- Reversal of strings.
- Checking validity of brackets.
- Converting infix to postfix and prefix.
- Evaluating the postfix expression.

Checking expression validity

Total number of left parenthesis should be equal to number of right parenthesis in the expression.

For every right parenthesis in the expression there should be left parenthesis of the same type.

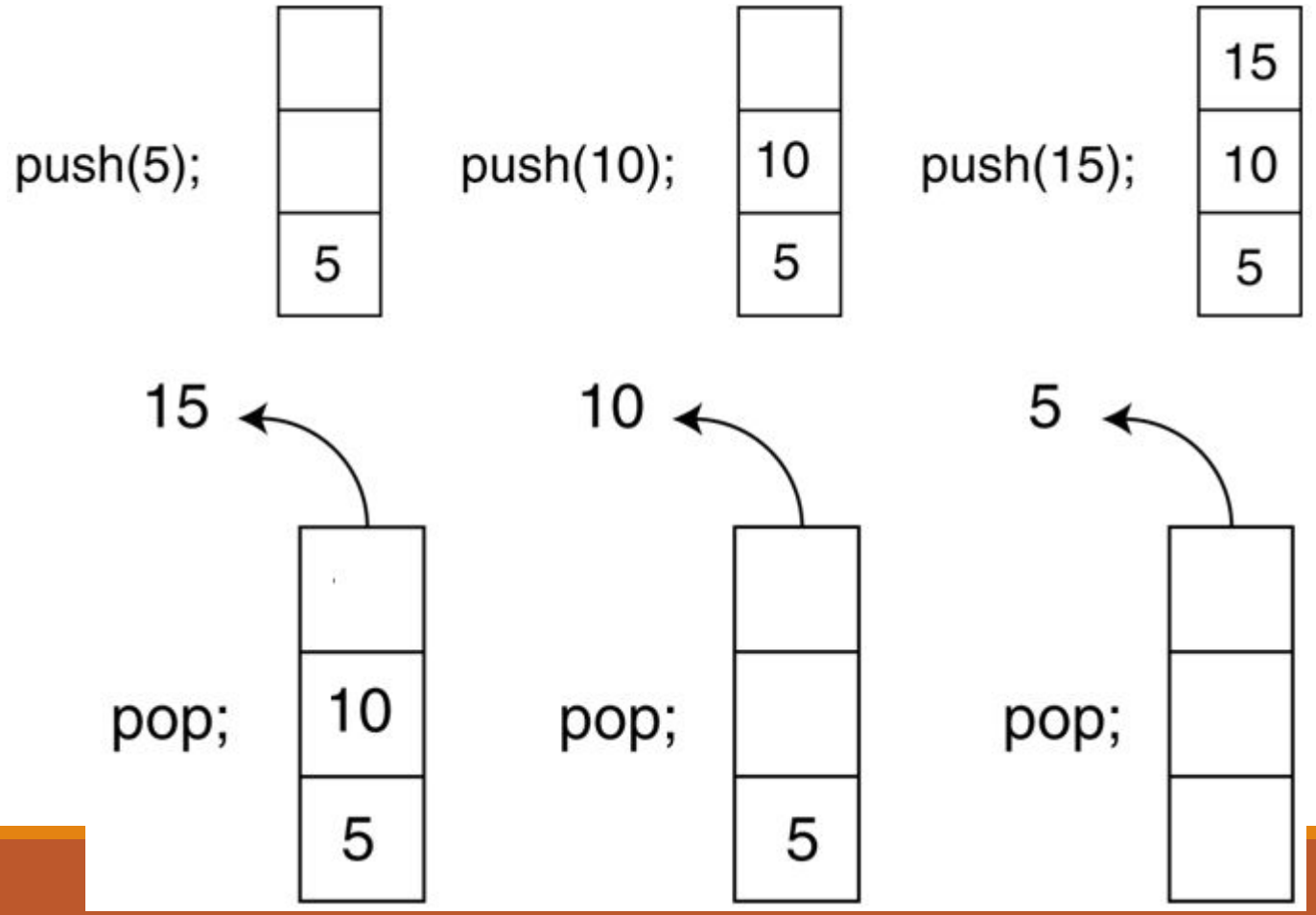
For example:

[19-5*(9+4)	**Invalid**
(1+5}	**Invalid**
[5+4*(9-2)]	**Valid**

Algorithm for reversal of strings

- Create an empty stack.
- One by one push all characters of string to stack.
- One by one pop all characters from stack and put them back to string.

Push and Pop Operations on Stack

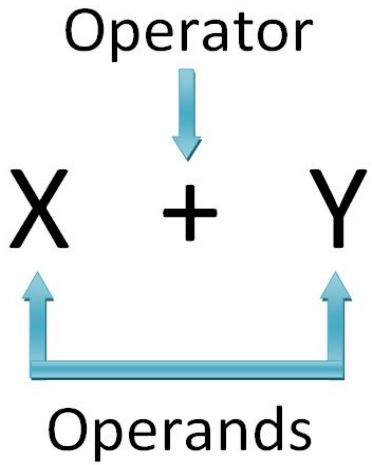


Reversal of string

Top/Index	Data
4	A
3	I
2	R
1	A
0	M

Top/Index	Data
4	M
3	A
2	R
1	I
0	A

Algebraic Expression



An algebraic expression is a legal combination of operands and the operators.

- Operand is the quantity on which a mathematical operation is performed.
- Operator is a symbol which signifies a mathematical or logical operation.

Infix, Postfix and Prefix Expressions

An **INFIX:** expressions in which operands surround the operator. E.g. A+B

POSTFIX: operator comes after the operands, also Known as Reverse Polish Notation (RPN). E.g. AB+

PREFIX: operator comes before the operands, also Known as Polish notation. E.g. +AB

Example

- Infix: A+B-C Postfix: AB+C- Prefix: -+ABC

Why do we need PREFIX/POSTFIX?

Appearance may be misleading, INFIX notations are not as simple as they seem

When you write an arithmetic expression such as B * C, the form of the expression provides you with information so that you can interpret it correctly.

In this case we know that the variable B is being multiplied by the variable C since the multiplication operator * appears between them in the expression. This type of notation is referred to as infix since the operator is in between the two operands that it is working on.

Why do we need PREFIX/POSTFIX?

Consider another infix example, A + B * C.

The operators + and * still appear between the operands, but there is a problem.

Which operands do they work on? Does the + work on A and B or does the * take B and C? The expression seems ambiguous.

Solve this problem using operator precedence rules

Why do we need PREFIX/POSTFIX?

Operators	Symbols	Associativity
Parenthesis	( ), {}, [ ]	Left to Right
Exponents	^	Right to Left
Multiplication and Division	*, /	Left to Right
Addition and Subtraction	+ , -	Left to Right

Operators of higher precedence are used before operators of lower precedence. The only thing that can change that order is the presence of parentheses. The precedence order for arithmetic operators places multiplication and division above addition and subtraction. If two operators of equal precedence appear, then a left-to-right ordering or associativity is used.

Category	Operator	Associativity
Postfix	() [] -> . ++ - -	Left to right
Unary	+ - ! ~ ++ - - (type)* & sizeof	Right to left
Multiplicative	* / %	Left to right
Additive	+ -	Left to right
Shift	<< >>	Left to right
Relational	< <= > >=	Left to right
Equality	== !=	Left to right
Bitwise AND	&	Left to right
Bitwise XOR	^	Left to right
Bitwise OR	|	Left to right
Logical AND	&&	Left to right
Logical OR	||	Left to right
Conditional	?:	Right to left
Assignment	= += -= *= /= %=>>= <<= &= ^= |=	Right to left
Comma	,	Left to right

Why do we need PREFIX/POSTFIX?

To evaluate an infix expression we need to consider

- Operators' Priority
- Associative property
- Delimiters

Why do we need PREFIX/POSTFIX?

Infix Expression Is Hard To Parse and difficult to evaluate.

Postfix and prefix do not rely on operator priority and are easier to parse.

An expression in infix form is thus converted into prefix or postfix form and then evaluated without considering the operators priority and delimiters.

In polish notation we don’t have a need of any parenthesis and by scanning from left to right we can evaluate the expression one by one. There is no need of traversal in expression at different places for evaluation.

Rules to convert infix to postfix expression

Parse from left to right

- Print the operand as they arrive.
- If the stack is empty or contains a left parenthesis on top, push the incoming operator on to the stack.
- If the incoming symbol is '(', push it on to the stack.
- If the incoming symbol is ')', pop the stack and print the operators until the left parenthesis is found.
- If the incoming symbol has higher precedence than the top of the stack, push it on the stack.

Rules to convert infix to postfix expression

- If the incoming symbol has lower precedence than the top of the stack, pop and print the top of the stack. Then test the incoming operator against the new top of the stack.
- If the incoming operator has the same precedence with the top of the stack then use the associativity rules.
  - If the associativity is from left to right then pop and print the top of the stack then push the incoming operator.
  - If the associativity is from right to left then push the incoming operator.
- At the end of the expression, pop and print all the operators of the stack.

A+B*C in postfix

Applying the rules of precedence, we obtained

A+B*C

A+(B*C) Parentheses for emphasis

A+(BC*) Convert the multiplication,

ABC*+ Postfix Form

Examples of infix to prefix and post fix

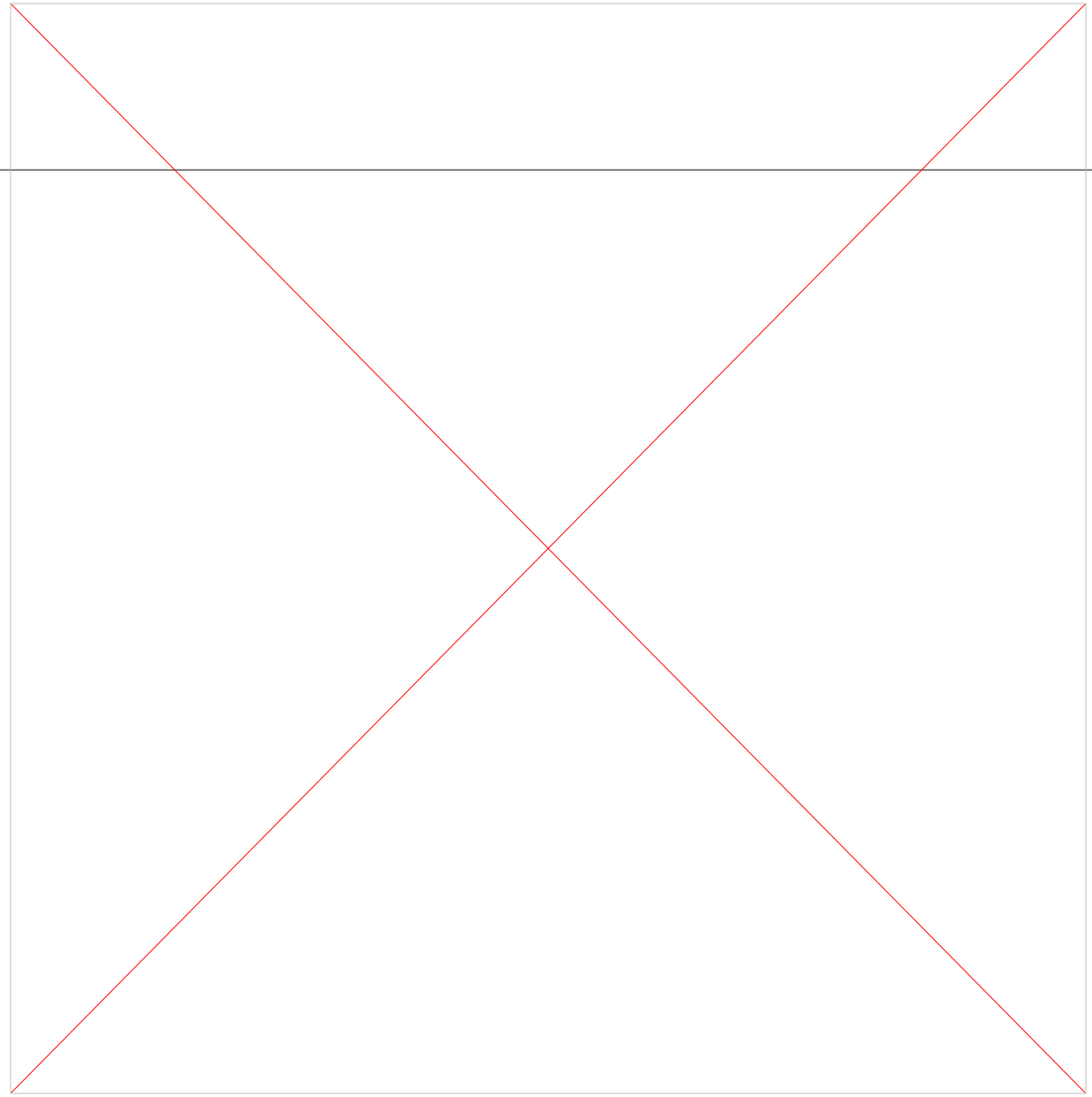
Infix	PostFix
A+B	AB+
(A+B) * (C + D)	AB+CD+*
A-B/(C*D^E)	ABCDE^*/-

Infix to Postfix Conversion

The steps in converting the infix expression a / b * (c + (d - e)) to postfix form

Next Character from Infix Expression	Postfix Form	Operator Stack (bottom to top)
a	*a*	
/	*a*	/
b	*a b*	/
*	*a b /*	
	a b /	*
(	*a b /*	* (
c	*a b / c*	* (
+	*a b / c*	* (+
(	*a b / c*	* (+ (
d	*a b / c d*	* (+ (
–	*a b / c d*	* (+ (–
e	*a b / c d e*	* (+ (–
)	*a b / c d e –*	* (+ (
	a b / c d e –	* (+
)	*a b / c d e – +*	* (
	a b / c d e – +	*
	*a b / c d e – + **	

Infix to Postfix using stacks



Infix to Postfix

Expression : A + B * C / D - F + A ^ E

Scanned Symbol	Stack	Output	Reason	
A		A	Step 2	
+	+	A	Step 3.1	
B	+	AB	Step 2	
*	+*	AB	Step 3.1	
C	+*	ABC	Step 2	
/	+/	ABC*	Step 3.2	/ prec is equal to * So not higher, thus going to Step 3.2
D	+/	ABC*D	Step 2	
-	-	ABC*D/+	Step 3.2	/ will be popped, added to o/p & then + popped & added to o/p, then - will be pushed
F	-	ABC*D/+F	Step 2	
+	+	ABC*D/+F-	Step 3.2	- will be popped, added to the o/p and then + added to the stack
A	+	ABC*D/+F-A	Step 2	
^	+^	ABC*D/+F-A	Step 2	
E	+^	ABC*D/+F-AE	Step 2	
(Empty)		ABC*D/+F-A^+	Step 8	

Yours Turn

(A+B)/E*F+C

A*(B+D)/E-F*(G+H/K)

Q Translate infix expression into its equivalent post fix expression:
(A-B)*(D/E)

Ans: (A-B)*(D/E) = [AB-]*[DE/] = AB-DE/*

Q. Translate infix expression into its equivalent post fix expression:
(A+B^D)/(E-F)+G

Ans : (A+B^D)/(E-F)+G = (A+BD^])/[EF-]+G = [ABD^+]/[EF-]+G

= [ABD^+EF-/]+G = ABD^+EF-/G+

Q.Translate infix expression into its equivalent post fix expression:

A*(B+D)/E-F*(G+H/K)

Ans: A*(B+D)/E-F*(G+H/K) = A*[BD+]/E-F*(G+[HK/])

[ABD+*]/E-F*[GHK/+]

[ABD+*E/]-[FGHK/+*] = ABD+*E/FGHK/+*-

Algorithm for evaluation of postfix expression

Scan the expression from left to right.

- If operand is encountered, then push it onto stack.
- If operator is encountered, then pop the last two elements from stack.

  b=stack[top], a=stack[top-1], c=a operator b.

  Push the result ‘c’ back to stack.

Result is equal to the top value of stack.

Algorithm for evaluation of postfix expression

Let the given expression be “456*+“. We scan all elements one by one.

Step	Input Symbol	Operation	Stack	Calculation
1.	4	Push	4	
2.	5	Push	4,5	
3.	6	Push	4,5,6	
4.	*	Pop(2 elements) & Evaluate	4	5*6=30
5.		Push result(30)	4,30	
6.	+	Pop(2 elements) & Evaluate	Empty	4+30=34
7.		Push result(34)	34	
8.		No-more elements(pop)	Empty	34(Result)

Questions?